NKIOLIB API user manual

Version: 5.0.0

Date: 23/12/05

Author: EA

NKIOLIB API user manual

5.0.0

2023-12-5

Distribution list:

Name / Group	Company
EA	

Reviews/Approvals:

	Name / Function / Company	Signature
Author:	EA	
Reviewed by:		

Table of Contents:

1Histrory

Version	Date	Author	Description
1.0.0	2020-7-10	EA	Initial version
4.0.0	2020-12-16	EA	For hardware version 4.0
4.0.1	2020-12-21	EA	Fix some bug, add the C# example
4.1.0	2021-01-28	EA	Optimize the API to be compatible with hardware version 2.x
4.1.5	2021-04-13	EA	Add the API description for the holding time unit of the light control which is only available for the firmware version from 4.2.3.
4.1.6	2021-04-18	EA	Change DIO function description: for the DO, set 0 to turn on and set 1 to turn off.
4.1.7	2021-05-12	EA	Add the APIs to read back the digital output by bit, byte and word in the polling mode. See Polling Mode.
4.1.8	2021-07-27	EA	Add the light control mode contents. See LC_SetPwmParams
4.1.9	2021-08-20	EA	Add the API to turn on/off the light without save. See LC_SetPwmParamsNoRetentive
5.0.0	2023-12-01	EA	Modify the library.Separate DIO and light control into two libraries.This library is for DIO.

2Introduction

Nodka NP-61xx Automation PC series products provide the interface to extend the functionalities by the add-on board.
For the DIO features, Nodka provides the dynamic library and export the function interfaces to be called by the user's program to access the data.

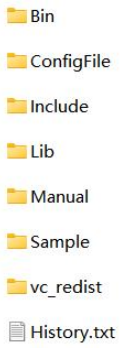
The digital I/O function will be illustrated in this document. Currently, the library and driver supports the below OS:

- Windows 7 (32-bit & 64-bit)
- Windows 8 (32-bit & 64-bit)
- Windows 10 (32-bit & 64-bit)

For the other OS to be supported, please contact Nodka support for the further information.

3Installation

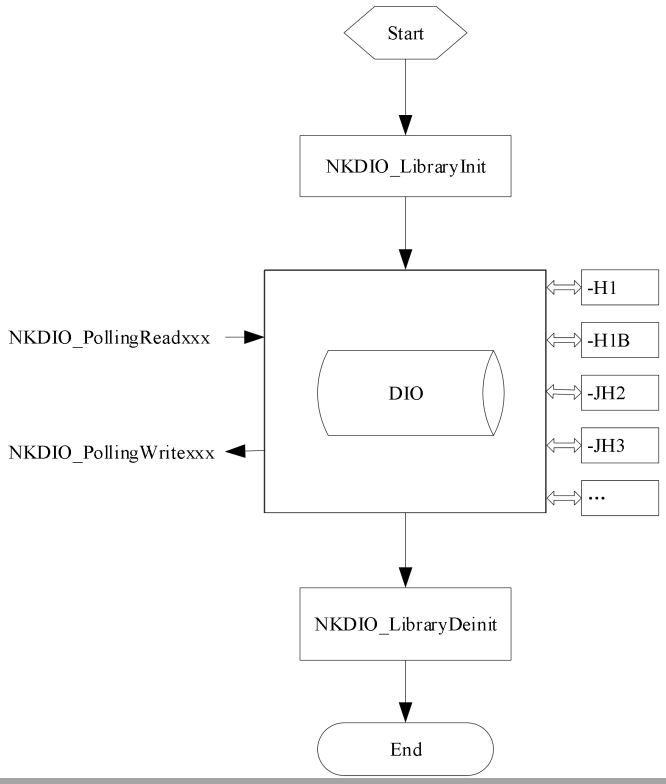
The drivers and library files can be decompressed by installing the 'NKDIOLC_Driver_Setup_x86.exe' file. The default installation path is 'C:\NODKA', a folder named NKDIOLC_SDK will be found after the driver installed successfully. The below is the contents list in the NKDIOLC_SDK folder.



- Bin: Test tool working folder, which can be used to test the functionality without coding.
- ConfigFile:Configuration files for different models.
- Include: the header files including the API declaration, which can be included in the C/C++ development.
- Lib: library bineary files which will be linked to the execute files.
- Manual: user manual for the SDK.
- Sample: some examples to be referenced during the development.
- vc_redist: VC++ runtime library, you must install them manually if your PC has not installed.

4API usage

The dynamic library can be used to access many add-on boards, the library must be initialized before connecting to the device and accessing the data, and library process function must be called cyclically to execute the command and communicate with the devices. There is a profile file which stored the configure information for the board, which must be imported during the library initialization. The blow is the logic schematic for your reference.



4.1API function list

Function Name	Description
DIO Library	
DIO Library Base	
NKDIO_LibraryInit	Init the DIO library and the drivers
NKDIO_LibraryDeinit	Deinitialize the DIO library and release the resources
DIO Functions	
NKDIO_PollingReadDiByte	Read the DI port in the polling mode
NKDIO_PollingReadDiWord	Read two DI ports in the polling mode
NKDIO_PollingWriteDoByte	Write the DO port in the polling mode
NKDIO_PollingWriteDoWord	Write two DO ports in the polling mode
NKDIO_PollingReadDoByte	Read back the set value of the digital output port(8 bits).
NKDIO_PollingReadDoWord	Read back the set value of the digital output port(16 bits).

4.2DIO Library

4.2.1DIO Library Base

4.2.1.1NKDIO_LibraryInit

- **Description**
  DIO Library initialized.
- **Prototype**

int NKDIO_LibraryInit(const char * configFile)

- **Parmeters**
  - Input
    - configIniFile：the name and path of the configure file. The configuration for the IO for each add-on board is stored in the ini format file, which can be got in the according folder after the SDK is installed.
  - Output
    - None
- **Return**
  Return 0 if success; other return error codes.
- **Other**
  The function should be called firstly but only once to allocate the resources before using the I/O access functions in the library.
- **Usage**
  int ret = NKDIO_LibraryInit (“./nkio_config.ini”);

4.2.1.2NKDIO_LibraryDeinit

- **Description**
  The function to be used to release the DIO library resources before the application exit.
- **Prototype**
  void NKDIO_LibraryDeinit(void)
- **Parmeters**
  - Input
    - None
  - Output
    - None
- **Return**
  None.
- **Other**
  The function should be called before the application exit.
- **Usage**

```
int ret = NKDIO_LibraryInit(“nkio_config.ini”);
if( ret == NKIO_ENOERR)
{
    While(1)
    {
       ……
      Sleep(1);
    }

}
NKDIO_LibraryDeinit();
```

The library provide the functions to access the digital I/O in the polling mode, but in the case of multi-thread access the independent I/O channel, the critical mutex must be used to protect the resources.

4.2.2DIO Functions

4.2.2.1NKDIO_PollingReadDiByte

- **Description**
  Read digital input port value(8 channels) .
- **Prototype**
  int NKDIO_PollingReadDiByte(unsigned char diByteIndex,unsigned char *pByteValue)
- **Parmeters**
  - Input
    - diByteIndex：The byte index of the digital input channel located, for the first 8 channel, the diByteIndex is set to 0, while the 8~15 channel is set to 1.
  - Output
    - pByteValue**:** Pointer to store the read data

- **Return**
  Return 0 if success; other return error codes.
- **Other**
  This function must be used after the DIO port is initialized.
- **Usage**

```
unsigned char port0Value = 0;
unsigned char port0Index = 0;
int ret = NKIO_ENOERR;
if(NKIO_ENOERR == NKDIO_LibraryInit("./nkio_config.ini"))
{
    ret = NKDIO_PollingReadDiByte(port0Index, &port0Value);  // Read data from port0.
    if(ret != NKIO_ENOERR )
    {
       printf("Read Data Error!, ErrorCode: %d\n\r", ret);
       return ret;
    }
    else
    {
       printf("Read Data Success!, Port value: %d\n\r", port0Value);
       return ret;

    }
    ……
}
else
{
      printf("DIO library initialized Error!, ErrorCode: %d\n\r", ret);
       return ret;
}
```

4.2.2.2NKDIO_PollingReadDiWord

- **Description**
  Read digital input port value(16 channels) .
- **Prototype**
  int NKDIO_PollingReadDiWord(unsigned char diWordIndex, unsigned short *pWordValue)
- **Parmeters**
  - Input
    - diWordIndex：Index of the first DI port of both DI input ports, the first two ports set to 0.
  - Output
    - pWordValue**:** Pointer to store the read data.
- **Return**
  Return 0 if success; other return error codes.
- **Other**
  This function must be used after the DIO port is initialized.
- **Usage**

```
unsigned short wordValue = 0;
unsigned char wordIndex = 0;
int ret = NKIO_ENOERR;
if(NKIO_ENOERR == NKDIO_LibraryInit("./nkio_config.ini"))
{
    ret = NKDIO_PollingReadDiWord(wordIndex, & wordValue);  // Read data from the first 2 ports.
    if(ret != NKIO_ENOERR )
    {
       printf("Read Data Error!, ErrorCode: %d\n\r", ret);
       return ret;
    }
    else
    {
       printf("Read Data Success!, Word value: %d\n\r", wordValue);
       return ret;
```

```
    }
    ……
}
else
{
    printf("DIO library initialized Error!, ErrorCode: %d\n\r", ret);
    return ret;
}
```

4.2.2.3NKDIO_PollingWriteDoByte

- **Description**
  Write the output port value (8 channels).
- **Prototype**
  int NKDIO_PollingWriteDoByte(BYTE doByteIndex, BYTE doByteValue)
- **Parmeters**
  - Input
    - doByteIndex：The byte index of the digital output channel located, for the first 8 channel, the doByteIndex is set to 0, while the 8~15 channel is set to 1.
    - doByteValue: the port status to be set, from 0 to 0xFF. 0x0 to turn on all of the port, and set to 0xFF to reset the port.
  - Output
    - None
- **Return**
  Return 0 if success; other return error codes.
- **Other**
  This function must be used after the DIO port is initialized.
- **Usage**

```
unsigned char portIdx = 0;
unsigned char portValue = 0x0F; // turn on the bit4~8 in the first port.
if(NKIO_ENOERR == NKDIO_LibraryInit("./nkio_config.ini"))
{
    ret = NKDIO_ PollingWriteDoByte (portIdx, portValue);
    if(ret != NKIO_ENOERR )
    {
        printf("Write Data Error!, ErrorCode: %d\n\r", ret);
        return ret;
    }
    else
    {
        printf("Write Data Success!\n\r");
        return ret;

    }
    ……
}
else
{
    printf("DIO library initialized Error!, ErrorCode: %d\n\r", ret);
    return ret;
}
```

4.2.2.4NKDIO_PollingWriteDoWord

- **Description**
  Write the output port value (16 channels).
- **Prototype**
  int NKDIO_PollingWriteDoWord(unsigned char doWordIndex, unsigned short doWordValue)

- **Parmeters**
  - Input
    - doWordIndex：index of the first DO port of both DO input ports, the first two ports set to 0.
    - doWordValue:the port status to be set,, from 0x0000 to 0xFFFF. Open DDO channel according to bit set to 0, close DO channel set to 1, and the default value of DO port is 0 xFFFF.
  - Output
    - None
- **Return**

  Return 0 if success; other return error codes.
- **Other**

  This function must be used after the DIO port is initialized.
- **Usage**

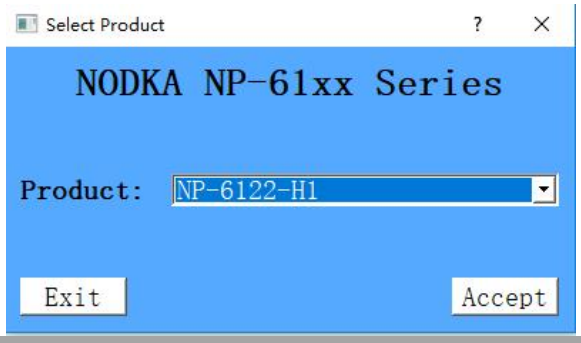
```
unsigned char Idx = 0;  // for the first two ports.
unsigned short wordValue = 0xFFFE; // turn on bit 0 in the first port.
if(NKIO_ENOERR == NKDIO_LibraryInit("./nkio_config.ini"))
{
    ret = NKDIO_ PollingWriteDoWord(Idx, wordValue);
    if(ret != NKIO_ENOERR )
    {
       printf("Write Data Error!, ErrorCode: %d\n\r", ret);
       return ret;
    }
    else
    {
       printf("Write Data Success!\n\r");
       return ret;

    }
    ……
}
else
{
      printf("DIO library initialized Error!, ErrorCode: %d\n\r", ret);
      return ret;
}
```

1 Test Tool

Target to easily check the IO board firstly before your development, Nodka provide a small tool called NKIO_TEST_TOOL.exe which can be used to check the IO working status, it is also can be used to configure some parameters when the IO light controller is working in the external trigger mode. Furthermore, the tool also provides the interface to update the firmware if necessary. The usage of the test tool will be illustrated in this chapter.

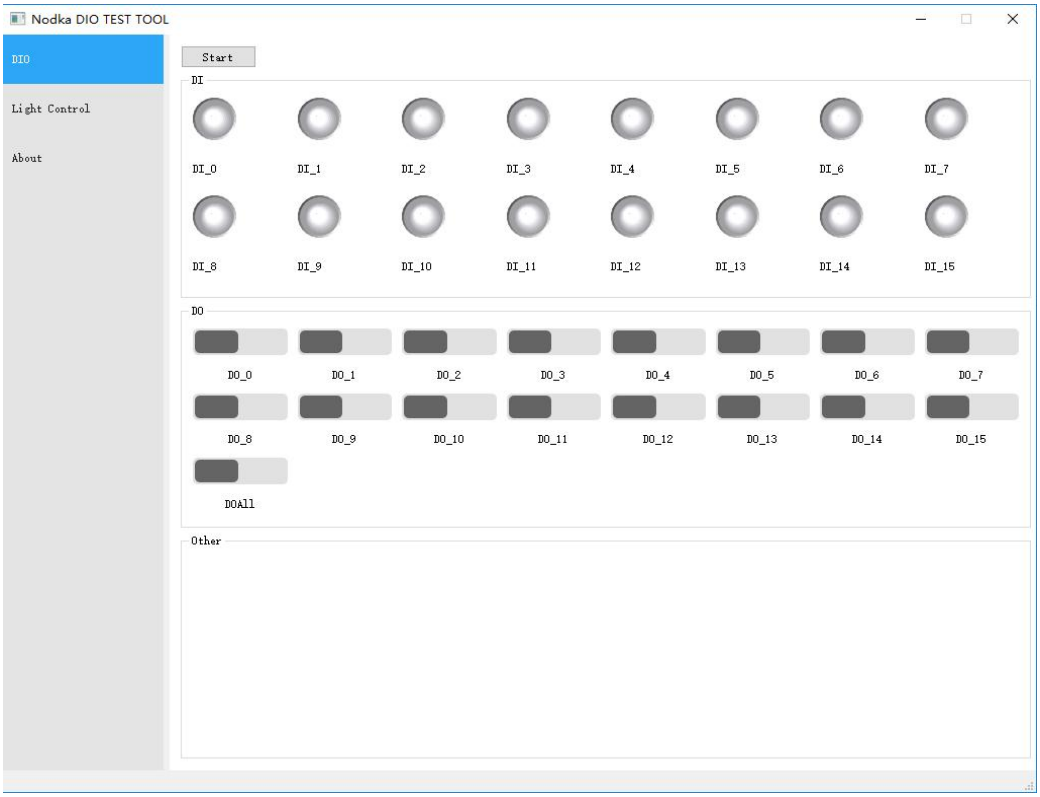
Since the test tool can be used for different I/O addon boards by importing the configuration file, so you must select correct board in the below dialog to import the according profile into the driver.



1.1 IO Test

The PC access the digital I/O by writing the value to the according registers directly, and in the I/O test page, one timer is used to refresh the I/O status cyclically after the start button is pushed. The Led will be change to green when the according input is on, and will reset to gray when the digital input is off.

There is one switch button for each digital output channel, you can press the according button to change the digital output status. But for the addon board which only support 8 channels digital output, only the first 8 switch buttons are available, and the others will make no sense when you change the status.



2 Development

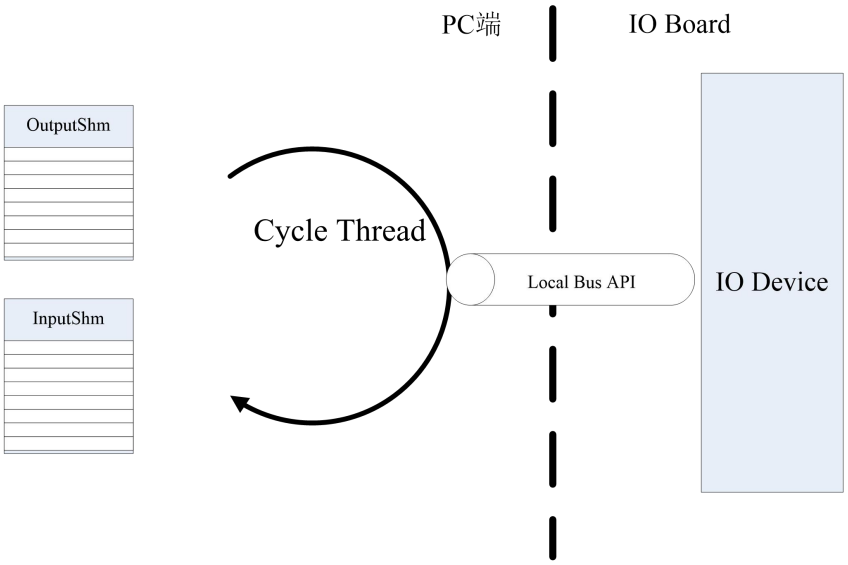
Nodka provide the dynamic library for the user to develop the programs.You can find the library and the header files in the SDK installation path. Please add the the files in the library folder into the same path with your own application working path.

Please refer to the according sample projects for the detailed information.

3 FAQ

3.1 How to access the DIO channels in the multi-thread?

Sine the IO devices can only be access by only one thread at a time, so the proposed solution is jus create a higher priority server thread as a master to refresh the DI and DO between the shared memory and the IO device, the other threads only need to read the data from the shared memory or write the data to the shared memory, as the below shows.



3.2 How to get the configure profile for different IO board?

The configuration of the IO board is illulstrated in the ini file. In the path of the SDK installation, the path of configure profile is listed in the config.ini file, you must import the according DIO configure profile in the DIO_Init function during development.